

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.

rehmang110_model_weight_path = kagglehub.dataset_download('rehmang110/model-weight')
rehmang110_3ddatasetfaces_path = kagglehub.dataset_download('rehmang110/3ddatasetfaces')
rehmang110_smoothfacedataet_path = kagglehub.dataset_download('rehmang110/smoothfacedataet')

print('Data source import complete.')
```

```
import numpy as np
import torch
import tqdm as tqdm
from PIL import Image
from pathlib import Path
import shutil
```

```
!pip install torchaudio --quiet
!pip install o imageio dlib cython cmake pyrallis onnx onnxruntime open3d
```

```
Requirement already satisfied: o in /usr/local/lib/python3.11/dist-packages (0.0.2b2)
Requirement already satisfied: imageio in /usr/local/lib/python3.11/dist-packages (2.37.0)
Requirement already satisfied: dlib in /usr/local/lib/python3.11/dist-packages (19.24.6)
Requirement already satisfied: cython in /usr/local/lib/python3.11/dist-packages (3.0.12)
Requirement already satisfied: cmake in /usr/local/lib/python3.11/dist-packages (3.31.6)
Requirement already satisfied: pyrallis in /usr/local/lib/python3.11/dist-packages (0.3.1)
Requirement already satisfied: onnx in /usr/local/lib/python3.11/dist-packages (1.18.0)
Requirement already satisfied: onnxruntime in /usr/local/lib/python3.11/dist-packages (1.22.1)
Requirement already satisfied: open3d in /usr/local/lib/python3.11/dist-packages (0.19.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from imageio) (1.26.4)
Requirement already satisfied: pillow>=8.3.2 in /usr/local/lib/python3.11/dist-packages (from imageio) (11.2.1)
Requirement already satisfied: typing-inspect in /usr/local/lib/python3.11/dist-packages (from pyrallis) (0.9.0)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.11/dist-packages (from pyrallis) (6.0.2)
Requirement already satisfied: protobuf>=4.25.1 in /usr/local/lib/python3.11/dist-packages (from onnx) (6.31.1)
Requirement already satisfied: typing_extensions>=4.7.1 in /usr/local/lib/python3.11/dist-packages (from onnx) (4.
Requirement already satisfied: coloredlogs in /usr/local/lib/python3.11/dist-packages (from onnxruntime) (15.0.1)
Requirement already satisfied: flatbuffers in /usr/local/lib/python3.11/dist-packages (from onnxruntime) (25.2.10)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from onnxruntime) (25.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.11/dist-packages (from onnxruntime) (1.13.1)
Requirement already satisfied: dash>=2.6.0 in /usr/local/lib/python3.11/dist-packages (from open3d) (3.2.0)
Requirement already satisfied: werkzeug>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from open3d) (3.1.3)
Requirement already satisfied: flask>=3.0.0 in /usr/local/lib/python3.11/dist-packages (from open3d) (3.1.1)
Requirement already satisfied: nbformat>=5.7.0 in /usr/local/lib/python3.11/dist-packages (from open3d) (5.10.4)
Requirement already satisfied: configargparse in /usr/local/lib/python3.11/dist-packages (from open3d) (1.7.1)
Requirement already satisfied: ipywidgets>=8.0.4 in /usr/local/lib/python3.11/dist-packages (from open3d) (8.1.5)
Requirement already satisfied: addict in /usr/local/lib/python3.11/dist-packages (from open3d) (2.4.0)
Requirement already satisfied: matplotlib>=3 in /usr/local/lib/python3.11/dist-packages (from open3d) (3.7.2)
Requirement already satisfied: pandas>=1.0 in /usr/local/lib/python3.11/dist-packages (from open3d) (2.2.3)
Requirement already satisfied: scikit-learn>=0.21 in /usr/local/lib/python3.11/dist-packages (from open3d) (1.2.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from open3d) (4.67.1)
Requirement already satisfied: pyquaternion in /usr/local/lib/python3.11/dist-packages (from open3d) (0.9.9)
Requirement already satisfied: plotly>=5.0.0 in /usr/local/lib/python3.11/dist-packages (from dash>=2.6.0->open3d)
Requirement already satisfied: importlib-metadata in /usr/local/lib/python3.11/dist-packages (from dash>=2.6.0->op
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from dash>=2.6.0->open3d) (2.3
Requirement already satisfied: retrying in /usr/local/lib/python3.11/dist-packages (from dash>=2.6.0->open3d) (1.4
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.11/dist-packages (from dash>=2.6.0->open3d)
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from dash>=2.6.0->open3d) (7
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.11/dist-packages (from flask>=3.0.0->open3
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.11/dist-packages (from flask>=3.0.0->open3d)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from flask>=3.0.0->
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.11/dist-packages (from flask>=3.0.0->open3d
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from flask>=3.0.0->op
Requirement already satisfied: comm>=0.1.3 in /usr/local/lib/python3.11/dist-packages (from ipywidgets>=8.0.4->ope
Requirement already satisfied: ipython>=6.1.0 in /usr/local/lib/python3.11/dist-packages (from ipywidgets>=8.0.4->
Requirement already satisfied: traitlets>=4.3.1 in /usr/local/lib/python3.11/dist-packages (from ipywidgets>=8.0.4
Requirement already satisfied: widgetsnbextension>=4.0.12 in /usr/local/lib/python3.11/dist-packages (from ipywidg
Requirement already satisfied: jupyterlab-widgets>=3.0.12 in /usr/local/lib/python3.11/dist-packages (from ipywidg
```

```
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3->open3d)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3->open3d)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3->open3d)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3->open3d)
Requirement already satisfied: pyparsing<3.1,>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3->open3d)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib>=3->open3d)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.11/dist-packages (from nbformat>=5.7.0->open3d)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.11/dist-packages (from nbformat>=5.7.0->open3d)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/local/lib/python3.11/dist-packages (from nbformat>=5.7.0->open3d)
Requirement already satisfied: mkl_fft in /usr/local/lib/python3.11/dist-packages (from numpy->imageio) (1.3.8)
```

```
!git clone https://github.com/rohit7044/3D-GANTex
```

Show hidden output

```
!gdown --id 1Yp01KfXvJHRmCBkErNa62dHm-CUjsoIk -O 3D-GANTex/ThreeDDFA_weights/weights/mb1_120x120.onnx
!ls -lh 3D-GANTex/ThreeDDFA_weights/weights/
```

Show hidden output

```
%cd /kaggle/working/3D-GANTex
!sh ./TDDFA_build.sh
%cd /kaggle/working/
```

```
/kaggle/working/3D-GANTex
/kaggle/working
```

```
src_path1 = Path("/kaggle/input/model-weight/restyle_e4e_ffhq.pt")
dest_path1 = Path("/kaggle/working/3D-GANTex/pretrained_models") / "restyle_e4e_ffhq.pt"
src_path2 = Path("/kaggle/input/model-weight/restyle_pSp_ffhq.pt")
dest_path2 = Path("/kaggle/working/3D-GANTex/pretrained_models") / "restyle_pSp_ffhq.pt"
shutil.copy(src_path1, dest_path1)
shutil.copy(src_path2, dest_path2)
if dest_path1.is_file() and dest_path2.is_file():
    print("Weights are placed in the folder pretrained wieghts.")
```

Weights are placed in the folder pretrained wieghts.

```
%cd /kaggle/working/3D-GANTex
```

```
/kaggle/working/3D-GANTex
```

```
!python main.py --input "input_data/00012.png" --pose-min -3 --pose-max 5 --center-pose 1 --output-dir "output" --
```

```
/usr/local/lib/python3.11/dist-packages/torch/utils/cpp_extension.py:2059: UserWarning: TORCH_CUDA_ARCH_LIST is not
If this is not desired, please set os.environ['TORCH_CUDA_ARCH_LIST'].
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/torch/utils/cpp_extension.py:2059: UserWarning: TORCH_CUDA_ARCH_LIST is not
If this is not desired, please set os.environ['TORCH_CUDA_ARCH_LIST'].
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/torch/onnx/utils.py:1828: UserWarning: No names were found for specified dyn
  warnings.warn(
/usr/local/lib/python3.11/dist-packages/torch/onnx/utils.py:1828: UserWarning: No names were found for specified dyn
  warnings.warn(
Convert /kaggle/working/3D-GANTex/FaceBoxes/weights/FaceBoxesProd.pth to /kaggle/working/3D-GANTex/FaceBoxes/weights
Loading ReStyle e4e from checkpoint: /kaggle/working/3D-GANTex/pretrained_models/restyle_e4e_ffhq.pt
Loading StyleGAN3 generator from path: None
Done!
Model successfully loaded!
Setting up PyTorch plugin "filtered_lrelu_plugin"... /usr/local/lib/python3.11/dist-packages/torch/utils/cpp_extens
If this is not desired, please set os.environ['TORCH_CUDA_ARCH_LIST'].
  warnings.warn(
Done.
Save visualization result to output/00012_uv_tex.png
Dump to output/00012_obj.obj
error: XDG_RUNTIME_DIR not set in the environment.
[Open3D WARNING] GLFW Error: Failed to detect any supported platform0;m
[Open3D WARNING] GLFW initialized for headless rendering.0;m
[Open3D WARNING] GLFW Error: OSMesa: Library not found0;m
[Open3D WARNING] Failed to create window0;m
[Open3D WARNING] [DrawGeometries] Failed creating OpenGL window.0;m
Processing completed successfully!
Generated files:
  pose: output/00012_pose.png
  uv_tex: output/00012_uv_tex.png
```

```
obj: output/00012_obj.obj
```

```
!ls -lh output
```

[Show hidden output](#)

```
!pip install trimesh
!pip install open3d
!pip install pymeshfix
```

[Show hidden output](#)

```
!pip install pymeshlab
```

[Show hidden output](#)

```
import numpy as np
from scipy import spatial
import trimesh
from pathlib import Path
import shutil
import open3d as o3d
```

```
import shutil
from pathlib import Path

c_path1 = Path("/kaggle/input/smoothfacedataet")
c_path2 = Path("/kaggle/input/3ddatasetfaces")
dest_path = Path("/kaggle/working/faceset/new")
dest_path.mkdir(parents=True, exist_ok=True)

for file in c_path2.iterdir():
    if file.is_file(): # copy only files, not subdirectories
        shutil.copy(file, dest_path / file.name)

for file in c_path1.iterdir():
    if file.is_file(): # copy only files, not subdirectories
        shutil.copy(file, dest_path / file.name)
```

Start coding or [generate](#) with AI.

```
import open3d as o3d
import numpy as np

def load_mesh_as_pointcloud(path, num_points=20000):
    mesh = o3d.io.read_triangle_mesh(path)
    mesh.compute_vertex_normals()
    pcd = mesh.sample_points_uniformly(number_of_points=num_points)
    return mesh, pcd

def center_mesh(mesh):
    center = mesh.get_center()
    mesh.translate(-center)
    return mesh

def normalize_scale(mesh, reference_mesh):
    bbox_mesh = mesh.get_max_bound() - mesh.get_min_bound()
    bbox_ref = reference_mesh.get_max_bound() - reference_mesh.get_min_bound()
    scale_factor = np.max(bbox_ref) / np.max(bbox_mesh)
    mesh.scale(scale_factor, center=np.zeros(3))
    return mesh

def run_icp(source_pcd, target_pcd, threshold=0.02, init_transform=np.identity(4)):
    reg = o3d.pipelines.registration.registration_icp(
        source_pcd, target_pcd, threshold, init_transform,
        o3d.pipelines.registration.TransformationEstimationPointToPoint())
    print(f"ICP fitness: {reg.fitness:.4f}, RMSE: {reg.inlier_rmse:.6f}")
```

```
return reg.transformation
```

```
# Load full head mesh first (used as reference for centering and scaling)
full_head_mesh, full_head_pcd = load_mesh_as_pointcloud('/kaggle/working/faceset/new/fullhead.obj')
full_head_mesh = center_mesh(full_head_mesh)
full_head_pcd = full_head_mesh.sample_points_uniformly(number_of_points=20000)

# Load head front area mesh and normalize scale relative to full head mesh
head_front_mesh, head_front_pcd = load_mesh_as_pointcloud('/kaggle/working/faceset/new/middlehead.obj')
head_front_mesh = center_mesh(head_front_mesh)
head_front_mesh = normalize_scale(head_front_mesh, full_head_mesh)
head_front_pcd = head_front_mesh.sample_points_uniformly(number_of_points=20000)

# Load front face mesh and normalize scale relative to full head mesh
front_face_mesh, front_face_pcd = load_mesh_as_pointcloud('/kaggle/working/faceset/new/frontface.obj')
front_face_mesh = center_mesh(front_face_mesh)
front_face_mesh = normalize_scale(front_face_mesh, full_head_mesh)
front_face_pcd = front_face_mesh.sample_points_uniformly(number_of_points=20000)
```

```
[Open3D INFO] Skipping non-triangle primitive geometry of type: 2
```

```
print("Aligning head front area mesh to full head mesh...")
tf_head_front_to_full_head = run_icp(head_front_pcd, full_head_pcd, threshold=0.05)
head_front_mesh.transform(tf_head_front_to_full_head)
head_front_pcd.transform(tf_head_front_to_full_head)

print("Aligning front face mesh to aligned head front area mesh...")
tf_front_face_to_head_front = run_icp(front_face_pcd, head_front_pcd, threshold=0.02)
front_face_mesh.transform(tf_front_face_to_head_front)
```

```
Aligning head front area mesh to full head mesh...
ICP fitness: 1.0000, RMSE: 0.007042
Aligning front face mesh to aligned head front area mesh...
ICP fitness: 1.0000, RMSE: 0.001048
TriangleMesh with 226755 points and 76073 triangles.
```

```
def project_and_adaptive_blend_mesh(source_mesh, target_mesh, max_alpha=0.9, min_alpha=0.1, smoothing_iterations=5):
    target_pcd = target_mesh.sample_points_uniformly(number_of_points=50000)
    kdtree = o3d.geometry.KDTreeFlann(target_pcd)

    source_vertices = np.asarray(source_mesh.vertices)
    new_vertices = []

    distances = []

    # First, collect distances to normalize blending factor later
    for v in source_vertices:
        _, idx, _ = kdtree.search_knn_vector_3d(v, 1)
        closest_point = np.asarray(target_pcd.points)[idx[0]]
        dist = np.linalg.norm(v - closest_point)
        distances.append(dist)
    distances = np.array(distances)
    max_dist = distances.max()
    min_dist = distances.min()

    for i, v in enumerate(source_vertices):
        _, idx, _ = kdtree.search_knn_vector_3d(v, 1)
        closest_point = np.asarray(target_pcd.points)[idx[0]]

        # Normalize distance between 0 and 1
        norm_dist = (distances[i] - min_dist) / (max_dist - min_dist + 1e-8)
        # Blend factor scales between min_alpha and max_alpha
        alpha = min_alpha + norm_dist * (max_alpha - min_alpha)

        blended_point = alpha * closest_point + (1 - alpha) * v
        new_vertices.append(blended_point)

    source_mesh.vertices = o3d.utility.Vector3dVector(np.array(new_vertices))
    source_mesh.compute_vertex_normals()

    smoothed_mesh = source_mesh.filter_smooth_laplacian(number_of_iterations=smoothing_iterations)
    smoothed_mesh.compute_vertex_normals()
```

```
return smoothed_mesh
```

```
# Assume head_front_mesh and detailed_front_face_mesh are aligned Open3D TriangleMesh objects
```

```
updated_head_front_mesh = project_and_blend_mesh(head_front_mesh, front_face_mesh)
```

```
import open3d as o3d
import numpy as np
from scipy.spatial import cKDTree

# Make sure meshes have vertex normals computed
full_head_mesh.compute_vertex_normals()
updated_head_front_mesh.compute_vertex_normals()

# Convert vertices to numpy arrays
full_head_vertices = np.asarray(full_head_mesh.vertices)
front_vertices = np.asarray(updated_head_front_mesh.vertices)

# Build KDTree on full head vertices for nearest neighbor search
kdtree = cKDTree(full_head_vertices)

# Find closest full head vertex for each front face vertex
distances, front_vertex_indices = kdtree.query(front_vertices)

print(f"Max vertex distance between front face and full head: {distances.max()}")

# Replace positions of these vertices in full head mesh
full_head_vertices[front_vertex_indices] = front_vertices

# Update full head mesh vertices
full_head_mesh.vertices = o3d.utility.Vector3dVector(full_head_vertices)

# Recompute normals for updated mesh
full_head_mesh.compute_vertex_normals()

# Save the merged mesh (adjust path)
output_path = '/kaggle/working/merged_fullhead.obj'
o3d.io.write_triangle_mesh(output_path, full_head_mesh)

print(f"Merged mesh saved to: {output_path}")
```

```
Max vertex distance between front face and full head: 0.016013122312243343
[Open3D WARNING] Write OBJ can not include triangle normals.0;m
[Open3D WARNING] Write PNG failed: image has no data.0;m
[Open3D WARNING] Write OBJ successful, but failed to write texture file.0;m
Merged mesh saved to: /kaggle/working/merged_fullhead.obj
```

```
import open3d as o3d
import numpy as np

# Assume you have your morphed mesh:
morphed_mesh = updated_head_front_mesh

# Sample a dense point cloud with normals from the mesh
pcd = morphed_mesh.sample_points_poisson_disk(number_of_points=50000)
pcd.estimate_normals(search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=0.1, max_nn=30))

# Run Poisson reconstruction with moderate depth (try 7 for less noise)
mesh_reconstructed, densities = o3d.geometry.TriangleMesh.create_from_point_cloud_poisson(pcd, depth=7)

# Convert densities to numpy array for filtering
densities = np.asarray(densities)

# Filter out vertices with low density (keep top 90%)
density_threshold = np.quantile(densities, 0.1)
vertices_to_keep = np.where(densities > density_threshold)[0]
mesh_filtered = mesh_reconstructed.select_by_index(vertices_to_keep)

# Crop mesh to bounding box of original point cloud to remove outliers/artifacts
bbox = pcd.get_axis_aligned_bounding_box()
```

```
mesh_cropped = mesh_filtered.crop(bbox)

# Apply mild Taubin smoothing to reduce roughness but preserve shape
mesh_smoothed = mesh_cropped.filter_smooth_taubin(number_of_iterations=5, lambda_filter=0.5, mu=-0.53)

# Compute normals again after smoothing
mesh_smoothed.compute_vertex_normals()

# Visualize improved mesh
o3d.io.write_triangle_mesh("results.ply", mesh_smoothed)
```

True

Start coding or [generate](#) with AI.

```
o3d.io.write_triangle_mesh("reconstructed_mesh.ply", mesh_cropped)
```

True